

1 **Part 7: Multivariate Time Series**

2 Text references: §13.4 in Ruppert and Matteson

3 Clearly, many financial time series have a strong correlation with one
4 another. One could use a regression approach to model this depen-
5 dence, but this assumes that there is a “response” that is to be pre-
6 dicted from a group of “covariates.” This is not always realistic or
7 desirable.

8 In this Part we will discuss models that will allow for joint mod-
9 elling of two or more time series. These are called *multivariate time*
10 *series models*. This is largely a discussion of how the ideas we have
11 discussed to this point generalize to the multivariate case.

Basic Definitions

Consider a multivariate time series $\{\mathbf{X}_t\}$ consisting of k series:

$$\mathbf{X}_t = \begin{bmatrix} X_{1t} \\ X_{2t} \\ \vdots \\ X_{kt} \end{bmatrix}$$

If the series $\{\mathbf{X}_t\}$ is *weakly stationary*, then $E(\mathbf{X}_t) = \boldsymbol{\mu}$ and $V(\mathbf{X}_t) = \boldsymbol{\Sigma}$ where $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ are not functions of t . Note that $\boldsymbol{\Sigma}$ need not be diagonal.

For a pair of time series, $\{X_{it}\}$ and $\{X_{jt}\}$, define the *cross-correlation* at lag h between the two series as

$$\rho_{i,j}(h) = \text{Corr}(X_{it}, X_{j(t-h)}).$$

Note that this is not a function of t , another consequence of assuming weak stationarity of the series.

Also note that $\rho_{i,j}(h) \neq \rho_{i,j}(-h)$ but that $\rho_{i,j}(h) = \rho_{j,i}(-h)$.

The definition of the *cross-covariance* is a natural extension; it is more typical to work in terms of the cross-correlation.

1 The *sample cross-correlation* between these series is defined as

$$2 \quad \hat{\rho}_{i,j}(h) = \frac{1}{n} \sum_{\ell=h+1}^n (X_{i\ell} - \bar{X}_i) (X_{j(\ell-h)} - \bar{X}_j) / (\widehat{\text{SD}}(X_i) \widehat{\text{SD}}(X_j))$$

3 for $h = 0, 1, \dots$. This function is returned by the R function `ccf()`.

4 The ordering of the series makes a difference, so it's important to
5 understand how it is handled. The following example can help:

```
6 x = ts(rnorm(1000))
```

```
7 lagx = lag(x, 5)
```

8 First, note how `lag()` behaves: It takes the original time series,
9 which is defined to start at time 1, and shifts it so that it starts at
10 time ~~-4~~ (as this is 5 time steps in the past).

11 Hence, there should be a strong (perfect) correlation between the
12 original series and the new series **five time steps in the past**.

13 In the notation we wrote above, $\hat{\rho}_{ij}(5)$ should be large if the original
14 series is the “ i ” series, and the lagged series is the “ j ” series.

- 1 Consider the output from `ccf()`:
- 2 `ccf(x, lagx, main="Sample CCF for Example")`

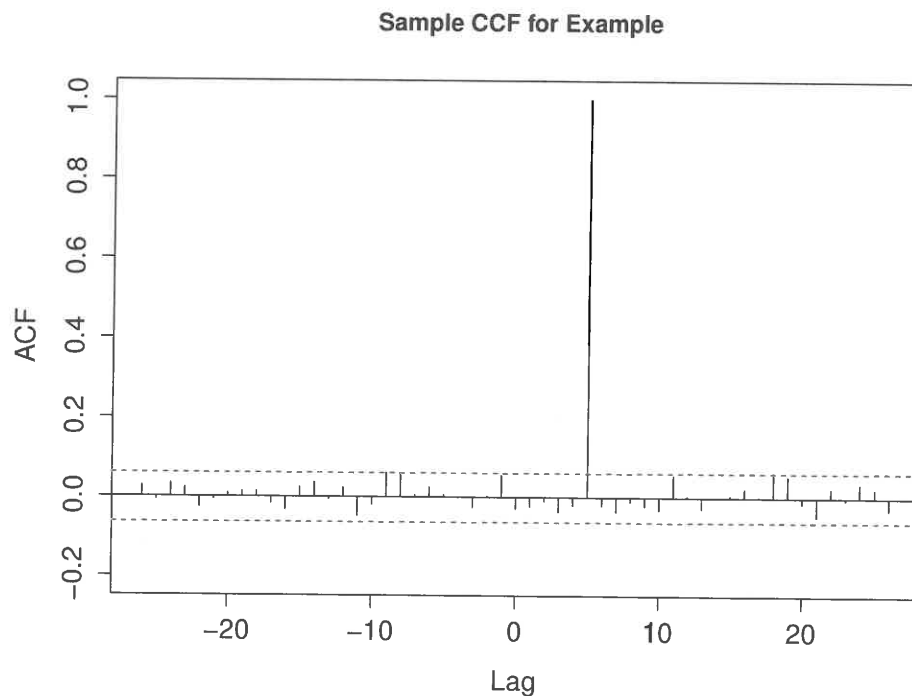


Figure 1: CCF for the simulated example.

- 3 From the spike at lag 5, it is clear that the ordering that R uses is
- 4 consistent with the ordering i, j used in our definition of $\hat{\rho}_{ij}$ above.

- 1 Consider the CCF shown as Figure 2. This compares the (monthly,
- 2 seasonally-adjusted) time series of consumer price index (CPI) and
- 3 industrial production (IP) from 2000 through 2017.

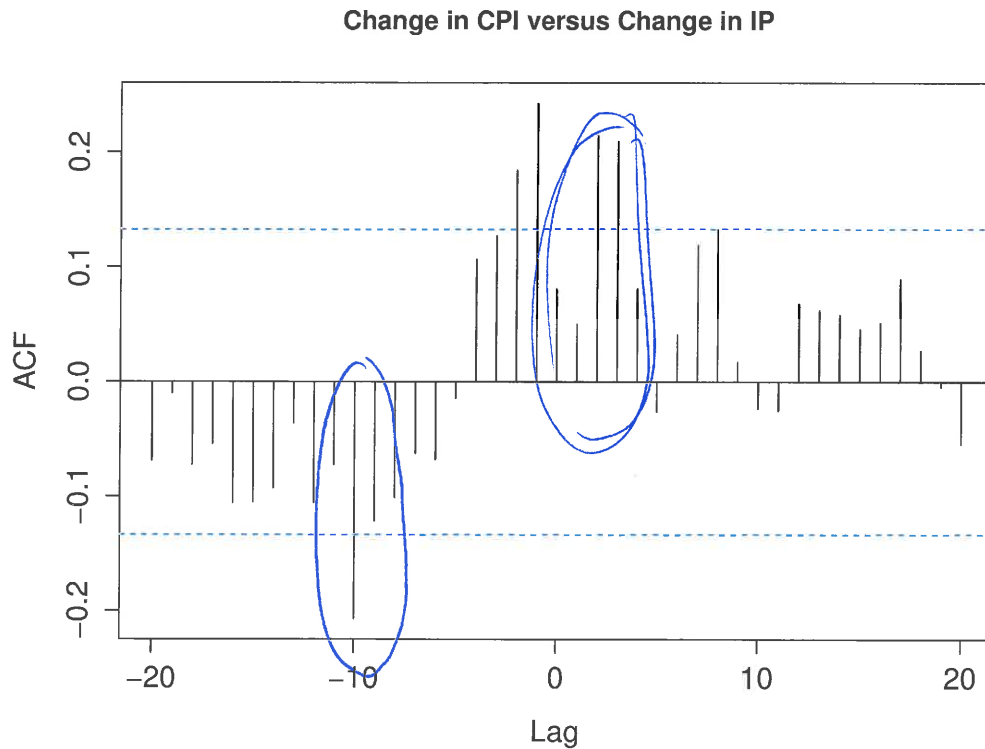


Figure 2: CCF comparing CPI and IP.

```

4 CPI = getSymbols("CPIAUCSL", src="FRED", auto.assign=F)
5 IP = getSymbols("INDPRO", src="FRED", auto.assign=F)
6
7 CPIdiff = diff(CPI)['2000::2017']
8 IPdiff = diff(IP)['2000::2017']
9
10 ccf(as.numeric(CPIdiff), as.numeric(IPdiff),
11     main="Change in CPI versus Change in IP",
12     cex.axis=1.3, cex.lab=1.3)

```

13

1 **Exercise:** Interpret Figure 2.

2 $|\hat{\rho}_{12}(-10)|$ is large. So, CPIdiff at
3 time t is correlated with IPdiff
4 at time $t - (-10) = t + 10$

6 $|\hat{\rho}_{12}(2)|$ and $|\hat{\rho}_{12}(3)|$ are large. So,
7 CPIdiff at time t is correlated
8 with IPdiff at time $t - 2, t - 3$

- 1 Again, consider that we have k series. The following notation is stan-
- 2 dard: The cross-covariances at lag h can be placed into a matrix Γ_h .
- 3 The cross-correlations are placed into a matrix ρ_h .
- 4 The sample versions are $\hat{\Gamma}_h$ and $\hat{\rho}_h$.
- 5 If the k series are loaded into the columns of a matrix `allseries`,
- 6 then `acf(allseries)` will return a k by k matrix of acf/ccf plots.
- 7 This capability is not built into `ccf()`.
- 8 See Figure 3 for this function applied to our CPI/IP data.

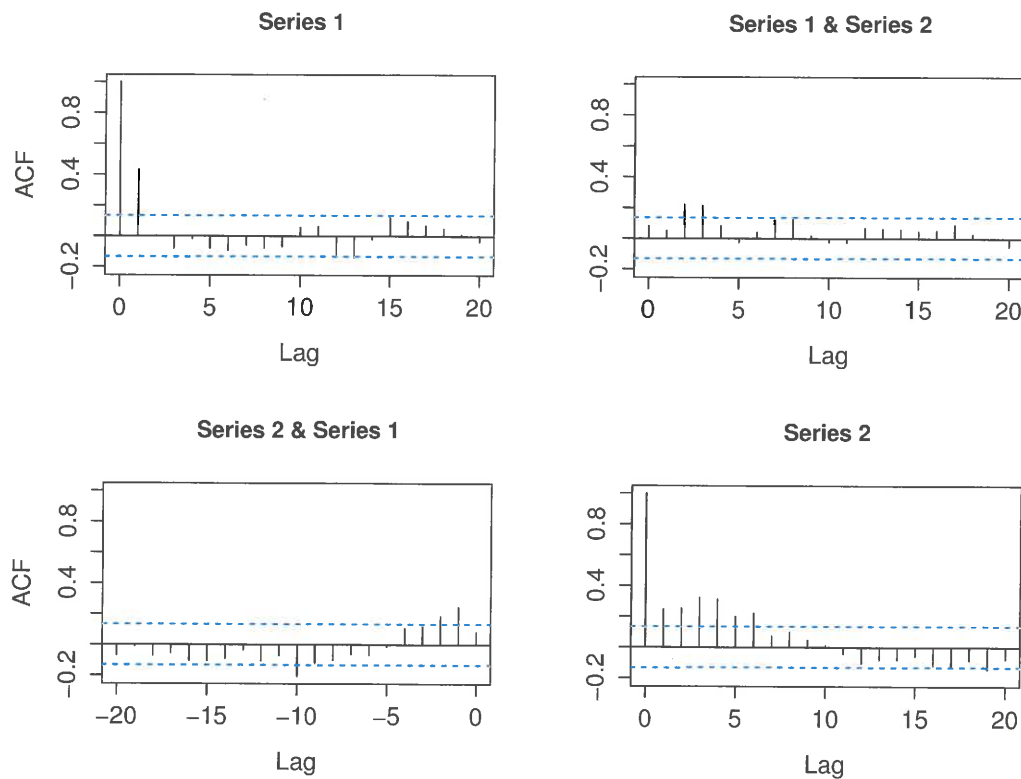


Figure 3: Function `acf()` applied to the (CPI, IP) data.

1 The Multivariate Ljung-Box Test

2 A multivariate version of the Ljung-Box Test is built into the R pack-
3 age `portes`; it is named `LjungBox()`.

4 This tests for any serial correlation at lags up to a specified limit,
5 both within a series, and between the provided series. The argument
6 `lags` allows for the specification for one or more maximum lag val-
7 ues.

```
8 library(portes)
9 LjungBox(cbind(as.numeric(CPIdiff), as.numeric(IPdiff)))
10 lags statistic df p-value
11    5   160.6330  20      0
12   10   207.6250  40      0
13   15   240.5382  60      0
14   20   268.8411  80      0
15   25   292.2442 100      0
16   30   317.3443 120      0
```

17 Not surprisingly, there is strong evidence of serial correlation in these
18 two series.

The VAR(p) Model

The *vector autoregressive (VAR) model* is a multivariate extension of the AR model. It can be written as follows:

$$\mathbf{X}_t = \boldsymbol{\mu} + \sum_{i=1}^p \boldsymbol{\Phi}_i (\mathbf{X}_{t-i} - \boldsymbol{\mu}) + \boldsymbol{\epsilon}_t$$

where $\boldsymbol{\epsilon}_t$ are uncorrelated vectors with mean zero and covariance $\boldsymbol{\Sigma}$.

The model is also sometimes written as

$$\mathbf{X}_t = \boldsymbol{\mu}' + \sum_{i=1}^p \boldsymbol{\Phi}_i \mathbf{X}_{t-i} + \boldsymbol{\epsilon}_t$$

where $\boldsymbol{\mu}' = (\mathbf{I} - \sum_{i=1}^p \boldsymbol{\Phi}_i) \boldsymbol{\mu}$.

It is worthwhile to note that $\mathbf{X}_t$, $\boldsymbol{\mu}$ and $\boldsymbol{\epsilon}_t$ are vectors of length k , and that $\boldsymbol{\Phi}_i$ is a k by k matrix.

Exercise: Recall that $\boldsymbol{\Sigma}$ need not be diagonal. What is the implication of this?

We think of $\boldsymbol{\epsilon}_t$ as the "innovations" at time t , the new info. at time t . The "new info" that arrives which influences series i may certainly ~~also~~ influence series j . Allowing entries of $\boldsymbol{\epsilon}_t$ to be correlated captures this dependence.

1 Consider the bivariate VAR(1) model. The model can be written as

2
$$X_{1t} = \mu_1 + \phi_{1,1}(X_{1(t-1)} - \mu_1) + \phi_{1,2}(X_{2(t-1)} - \mu_2) + \epsilon_{1t}$$

3 and

4
$$X_{2t} = \mu_2 + \phi_{2,1}(X_{1(t-1)} - \mu_1) + \phi_{2,2}(X_{2(t-1)} - \mu_2) + \epsilon_{2t}.$$

5 The power of the VAR model comes from the presence of $\phi_{1,2}$ and
6 $\phi_{2,1}$, allowing for the dependence of each series on the past values of
7 the other series.

8 **Technical Note:** The VAR(1) process is stationary provided all of the
9 eigenvalues of Φ are less than one in absolute value.

1 Example: VAR with R

2 Figure 4 shows the pair of series used above: monthly consumer
3 price index and industrial production for the period 2000 through
4 2017.

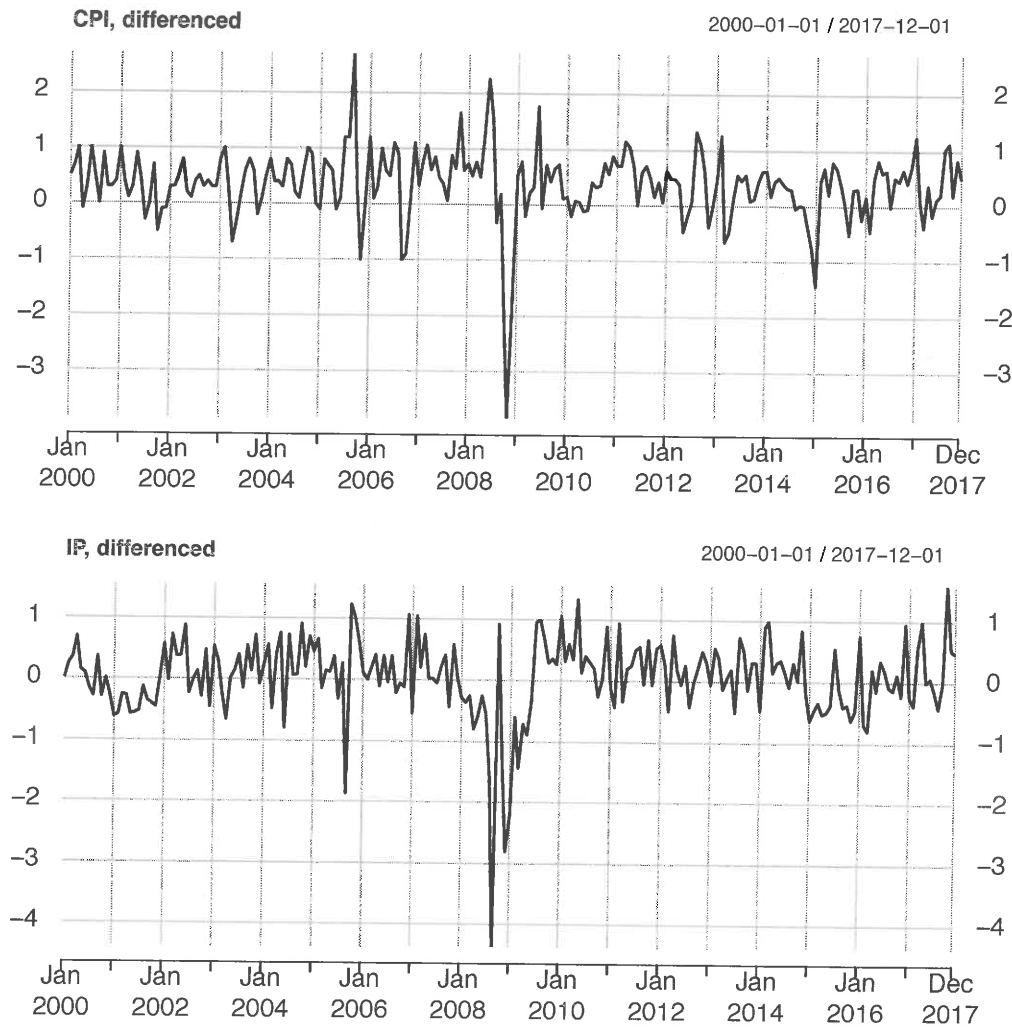


Figure 4: The differenced CPI, IP series.

5 We will fit this pair of series using a VAR model.

1 The commands used to obtain these data are as follows:

```
2 CPI = getSymbols("CPIAUCSL", src="FRED", auto.assign=F)
3 IP = getSymbols("INDPRO", src="FRED", auto.assign=F)
4
5 CPIdiff = diff(CPI) ['2000::2017']
6 IPdiff = diff(IP) ['2000::2017']
```

7 In R, VAR is implemented by the function `VAR()` as part of the pack-
8 age `vars`.¹ The syntax is simply as follows:

```
9 holdfit = VAR(cbind(CPIdiff, IPdiff), lag.max=10)
```

10 By setting `lag.max = 10`, the function uses AIC to select the order
11 p , considering models with $p \leq 10$.

12 Alternatively, one can fix the order using the argument `p`.

13 In this case, a model of order $p = 5$ is chosen.

¹VAR is also implemented as `ar()` as part of the standard distribution of R, but `ar()` does not allow for the easy construction of prediction intervals for forecasts.

- 1 The output of this function displays the fitted model. Note that this
 2 function uses the μ' parameterization.

3 Estimation results for equation CPIAUCSL:

4 =====

5 CPIAUCSL = CPIAUCSL.11 + INDPRO.11 + CPIAUCSL.12 +
 6 INDPRO.12 + CPIAUCSL.13 + INDPRO.13 +
 7 CPIAUCSL.14 + INDPRO.14 + CPIAUCSL.15 +
 8 INDPRO.15 + const

9
 10 Estimate Std. Error t value Pr(>|t|)

11	CPIAUCSL.11	0.49848	0.06961	7.161	1.49e-11	***	] first row of $\hat{\Phi}_1$
12	INDPRO.11	0.03720	0.06446	0.577	0.564530		
13	CPIAUCSL.12	-0.19966	0.07890	-2.530	0.012162	*	] $\hat{\Phi}_2$
14	INDPRO.12	0.23374	0.06372	3.668	0.000313	***	
15	CPIAUCSL.13	-0.06531	0.07979	-0.819	0.414027		
16	INDPRO.13	0.09641	0.06525	1.478	0.141061		
17	CPIAUCSL.14	0.05082	0.07794	0.652	0.515173		
18	INDPRO.14	-0.01553	0.06602	-0.235	0.814234		
19	CPIAUCSL.15	-0.15011	0.06925	-2.168	0.031373	*	
20	INDPRO.15	-0.11250	0.06592	-1.707	0.089458	.	
21	const	0.30228	0.05483	5.513	1.08e-07	***	] $\hat{\mu}_1$
22	---						

- 23 The "11" in the output is meant to indicate "at lag 1," and so forth.

1 The residuals in this case will be a collection of two-by-one vectors,
 2 one for each point in time. As before, there should not be any serial
 3 correlation in these residuals, but now we can look at both the ACFs
 4 for the individual series, and also the CCF between the two series.

5 In this case, the command to create Figure 5 is as follows.

```
6 acf(residuals(holdfit))
```

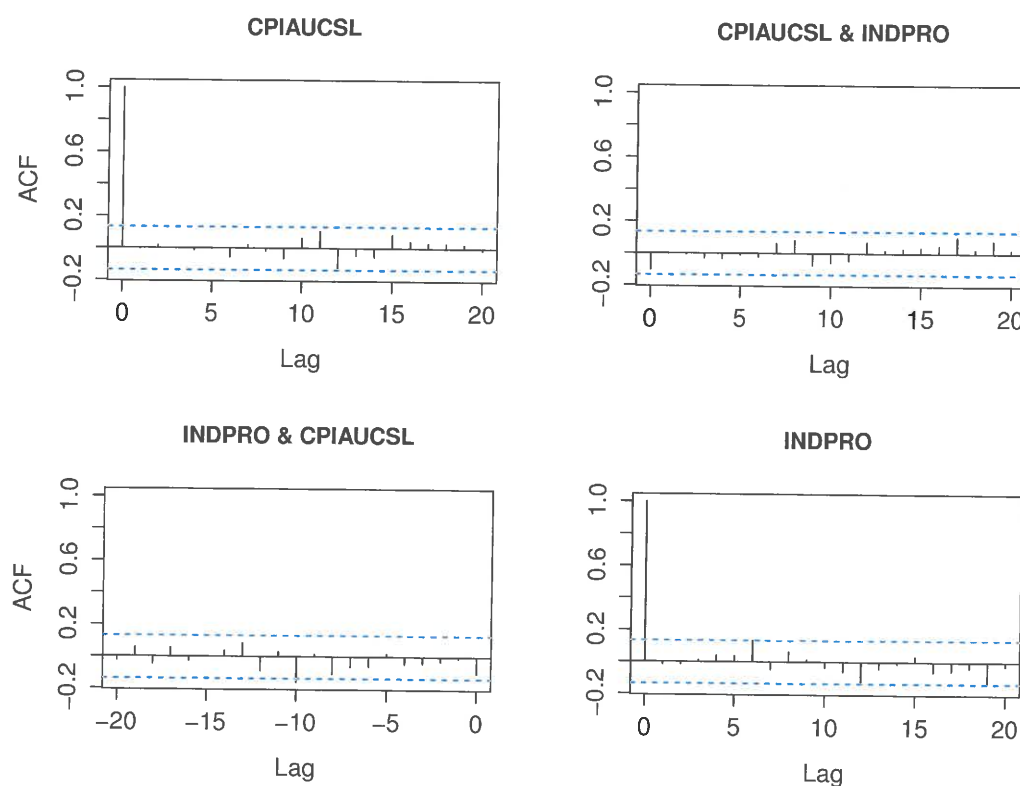


Figure 5: ACF and CCF of residuals from VAR fit.

7 The figure does not create of concern of remaining serial correlation.

1 The package `vars` also includes a function `serial.test()` which
2 is a multivariate version of the Ljung-Box test; this can be applied
3 to the output of `holdfit` in order to test for serial correlation in the
4 residuals.

```
5 > serial.test(holdfit)
```

```
6
```

```
7 Portmanteau Test (asymptotic)
```

```
8
```

```
9 data: Residuals of VAR object holdfit
```

```
10 Chi-squared = 44.523, df = 44, p-value = 0.4496
```

1 Forecasting with a VAR Model

2 Mathematically, the process of making forecasts with the VAR model
3 is much like with the AR model: replace parameters with their esti-
4 mates, and then project the values forward one step at a time.

5 The `predict()` function in R will work with the output of `VAR()`,
6 for instance:

```
7 > predict(holdfit,n.ahead=3)
8 $CPIAUCSL
9           fcst      lower      upper      CI
10 [1,] 0.6194043 -0.4472981 1.686107 1.066702
11 [2,] 0.4526141 -0.7380838 1.643312 1.190698
12 [3,] 0.3576980 -0.8692212 1.584617 1.226919
13
14 $INDPRO
15           fcst      lower      upper      CI
16 [1,] 0.4157636 -0.7449180 1.576445 1.160682
17 [2,] 0.4456237 -0.7263420 1.617589 1.171966
18 [3,] 0.4466196 -0.7384257 1.631665 1.185045
```

19 The default is to create 95% prediction intervals; this can be changed
20 by setting argument `ci`.

Consider a VAR(1) model.

Exercise: Show that the forecast for $\mathbf{X}_{n+h}$ is

$$\hat{\mathbf{X}}_{n+h} = \hat{\boldsymbol{\mu}} + \hat{\boldsymbol{\Phi}}^h (\mathbf{X}_n - \hat{\boldsymbol{\mu}})$$

for $h = 1, 2, \dots$

$$\mathbf{X}_{n+h} = \boldsymbol{\mu} + \boldsymbol{\Phi} (\mathbf{X}_{n+h-1} - \boldsymbol{\mu}) + \boldsymbol{\varepsilon}_{n+h}$$

When $h=1$

$$\mathbf{X}_{n+1} = \boldsymbol{\mu} + \boldsymbol{\Phi} (\mathbf{X}_n - \boldsymbol{\mu}) + \boldsymbol{\varepsilon}_{n+1}$$

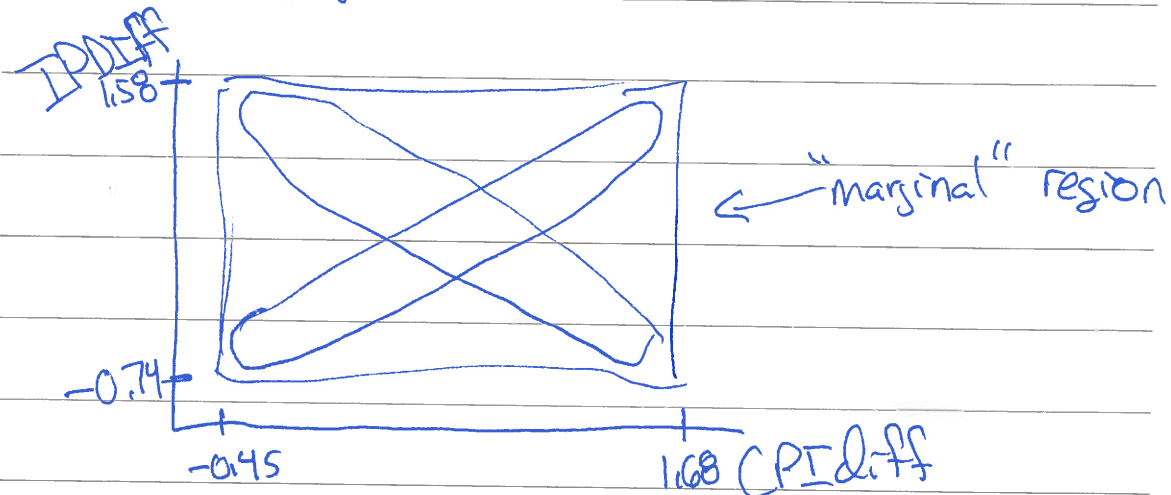
$$\hat{\mathbf{X}}_{n+1} = \hat{\boldsymbol{\mu}} + \hat{\boldsymbol{\Phi}} (\mathbf{X}_n - \hat{\boldsymbol{\mu}}) \neq \hat{\boldsymbol{\mu}}$$

When $h > 1$, get a recursive relationship just like in the 1D case,

$$\hat{\mathbf{X}}_{n+h} = \hat{\boldsymbol{\mu}} + \hat{\boldsymbol{\Phi}}^h (\mathbf{X}_n - \hat{\boldsymbol{\mu}})$$

- 1 **Exercise:** Why must care be taken in interpreting the intervals pro-
 2 duced by `predict()`?

3 The intervals produced by `predict()`
 4 are marginal.



11 It could be the case that strong correlation
 12 between the series mean that the prediction
 13 region is not rectangular, as focusing on
 the marginal intervals would imply

That correlation is captured in $\sum_{\tau} = V(\underline{\epsilon}_{\tau})$

1 This deficiency gives us an opportunity to introduce the use of sim-
2 ulations for this purpose. This technique is generally applicable,
3 but works particularly well in this type of situation where analyti-
4 cal forms are difficult to derive and/or compute.

5 Another advantage of this approach is that it avoids placing an as-
6 sumption on the distribution of the ϵ_i .

7 **Exercise:** Why is an assumption on the distribution of ϵ_i more diffi-
8 cult to justify in the multivariate case?

9 It is difficult to reliably estimate a
10 dist. in higher dimensions. Even if
11 you do fit a multivariate normal, it's
difficult to assess the quality of the fit.
Also, many parameters to estimate in
high-dimensional model.

1 Now we will consider the simulation process. As usual, it will sim-
2 plest to do this within the context of a $p = 1$ model.

3 First, note that the forecast for time $n + 1$ is

4
$$\hat{\mathbf{X}}_{n+1} = \hat{\boldsymbol{\mu}} + \hat{\boldsymbol{\Phi}}(\mathbf{X}_n - \hat{\boldsymbol{\mu}})$$

5 In the large sample approximation, the only difference between $\hat{\mathbf{X}}_{n+1}$
6 and $\mathbf{X}_{n+1}$ is due to $\boldsymbol{\epsilon}_{n+1}$.

7 We don't know $\boldsymbol{\epsilon}_{n+1}$, of course, but we can approximate a draw from
8 its distribution by **sampling from the residuals of the fit.**

“Bootstrap”

9 So, if I draw B times **with replacement** from the residuals (call these
10 $\hat{\boldsymbol{\epsilon}}_b^*$ for $b = 1, 2, \dots, B$) and then construct

11
$$\hat{\mathbf{X}}_{n+1,b}^* = \hat{\mathbf{X}}_{n+1} + \hat{\boldsymbol{\epsilon}}_b^* \quad \text{for } b = 1, 2, \dots, B.$$

12 **Exercise:** Explain how this is related to the *bootstrap* procedure.

13 bootstrap procedures approx. sampling from a
14 pop. with sampling with replacement from
15 a sample drawn from that population

The point is that our sample of $\hat{\mathbf{X}}_{n+1,b}^*$ will have (approximately) the same distribution as would have $\mathbf{X}_{n+1}$, and hence this sample can be used to construct an approximate prediction interval.

For instance, if `holdsamp` is a 2 by B matrix consisting of the simulated sample of values of $\hat{\mathbf{X}}_{n+1,b}^*$ then

`apply(holdsamp, 1, quantile, c(0.025, 0.975))`

will return the lower and upper endpoints of an approximate 95% prediction intervals for both components of $\mathbf{X}_{n+1}$.

$$g(X_{1,n+1}, X_{2,n+1})$$

This could easily be generalized to the case where $\mathbf{X}_{n+1}$ consists of k values.

Exercise: Explain how as k grows this simulation procedure avoids an increasingly complex estimation problem.

As k grows, fitting multivariate model
only becomes more challenging.

- 1 We can also extend this idea beyond the forecast of $\mathbf{X}_{n+1}$.
- 2 In fact, all we need to do is extend the B simulated values of $\hat{\mathbf{X}}_{n+1,b}^*$
- 3 enough steps until we reach the desired time point.
- 4 In other words, the exact same procedure is applied to obtain $\hat{\mathbf{X}}_{n+2,b}^*$
- 5 for $b = 1, 2, \dots, B$, and then again to obtain $\hat{\mathbf{X}}_{n+3,b}^*$ for $b = 1, 2, \dots, B$,
- 6 and so forth.

1 Some R code to implement the simulation procedure on our CPI/IP
2 example:

```
3 set.seed(0)
4 X = cbind(CPIdiff, IPdiff)
5 holdfitp1 = VAR(X, p=1)
6 B = 2000
7
8 muhat =
9   c(coef(holdfitp1)[[1]][3,1], coef(holdfitp1)[[2]][3,1])
10 Phihat =
11   rbind(coef(holdfitp1)[[1]][1:2,1], coef(holdfitp1)[[2]][1:2,1])
12
13 holdrows =
14   sample(1:nrow(residuals(holdfitp1)), size=B, replace=T)
15 epshat = residuals(holdfitp1)[holdrows,]
16
17 forecasts = muhat + Phihat %*% matrix(X[nrow(X),])
18 bootreps = matrix(rep(forecasts,B), ncol=2, byrow=T) + epshat
```

- 1 **R Exercise:** Update the code above to handle the case where predic-
2 tion bands are desired for 2 or more steps ahead.

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

12 _____

13 _____

14 _____

15 _____

16 _____

17 _____

18

1 Beyond the VAR Model

2 The ARMA model extends to the multivariate case without much
3 difficulty.

4 For example, the multivariate ARMA(1,1) model is written as

5
$$\mathbf{X}_t = \boldsymbol{\mu} + \boldsymbol{\Phi}(\mathbf{X}_{t-1} - \boldsymbol{\mu}) + \boldsymbol{\Theta}\boldsymbol{\epsilon}_{t-1} + \boldsymbol{\epsilon}_t$$

6 where $\boldsymbol{\Phi}$ and $\boldsymbol{\Theta}$ are both k by k matrices to be estimated.

7 In practice, however, the multivariate ARMA raises issues of com-
8 putational difficulty in obtaining the parameter estimates and non-
9 identifiability of the parameters. In addition, the VAR model has
10 proven to be sufficiently flexible to model most situations. For these
11 reasons, focus tends to remain on VAR models.

12 See Tsay, Chapter 8, for more details.

Multivariate Conditional Heteroskedastic Models

For a general multivariate time series $\mathbf{X}_t$, we can decompose into conditional mean and variance components, as before:

$$\mathbf{X}_t = \boldsymbol{\mu}_t + \mathbf{a}_t$$

where $\boldsymbol{\mu}_t$ is the conditional mean and $\boldsymbol{\Sigma}_t = \text{Cov}(\mathbf{a}_t | \mathcal{F}_{t-1})$.

An immediate problem that arises when trying to model $\boldsymbol{\Sigma}_t$ is the following: Since $\boldsymbol{\Sigma}_t$ is a k by k matrix, the number of parameters to be estimated will grow rapidly with k .

Models must also be constructed in such a way that they produce estimated covariance matrices which are positive definite.

1 Exponentially Weighted Moving Average Model

2 A simple model for the evolution of the covariance matrix is given
3 by the *exponentially weighted moving average (EWMA) model*.

4 Here, the covariance at time t can be written as a weighted com-
5 bination of the previous innovations $\mathbf{a}_t$ and the “prior” covariance
6 matrix:

$$\Sigma_t = (1 - \lambda) \mathbf{a}_{t-1} \mathbf{a}_{t-1}^T + \lambda \Sigma_{t-1} \quad \sum_{i=1}^{\infty} \lambda^i \mathbf{x}_i \mathbf{x}_i^T$$

8 There is a single parameter $0 < \lambda < 1$ to be estimated.

9 **Exercise:** Discuss the effect of varying λ in this model.

Typically $\lambda \approx 1$

If λ is smaller, then Σ_t is
greatly influenced by the magnitude
of $\mathbf{a}_{t-1}$

If λ is close to 1, the influence
of $\mathbf{a}_{t-1}$ is small

- 1 Ruppert and Matteson include a function `est.ewma()` for estimat-
2 ing λ . It can be accessed via the following command:

```
3 source("https://people.orie.cornell.edu/davidr/SDAFE2/Rscripts/SDAFE2.R")
```

- 4 Consider a matrix of daily returns:

```
5 ABT = getSymbols("ABT", from="2010-1-1", to="2018-6-30", auto=F)
6 CELG = getSymbols("CELG", from="2010-1-1", to="2018-6-30", auto=F)
7 LLY = getSymbols("LLY", from="2010-1-1", to="2018-6-30", auto=F)
8 JNJ = getSymbols("JNJ", from="2010-1-1", to="2018-6-30", auto=F)
9
10 logrets = cbind(dailyReturn(Ad(ABT), type="log"),
11                 dailyReturn(Ad(CELG), type="log"),
12                 dailyReturn(Ad(LLY), type="log"),
13                 dailyReturn(Ad(JNJ), type="log"))
14
15 logrets = logrets[-1,]
```

- 16 We find the MLE for λ , along with its standard error:

```
17 fitout = est.ewma(0.96, as.matrix(logrets))
18
19 fitout$lambda.hat
20 [1] 0.9908728
21
22 fitout$lambda.hat.se
23 [1,]
24 [1,] 0.001019273
```

← initial guess

1 One can also inspect the estimated covariance matrices using the

2 `function sigma.ewma()`:

```
3 allSigmas = sigma.ewma(fitout$lambda.hat, as.matrix(logrets))
```

```
4
```

```
5 allSigmas[, , 4]
```

```
6           [,1]           [,2]           [,3]           [,4]
```

```
7 [1,] 7.703657e-05 5.001685e-05 2.328372e-05 1.385430e-05
```

```
8 [2,] 5.001685e-05 3.223049e-04 1.059881e-04 1.093675e-04
```

```
9 [3,] 2.328372e-05 1.059881e-04 7.750994e-05 5.086369e-05
```

```
10 [4,] 1.385430e-05 1.093675e-04 5.086369e-05 6.575637e-05
```

11 It may be of interest to analyze how the variances and correlations

12 vary in time:

```
13 ser1 = xts(allSigmas[1,1,],time(logrets))[-c(1:20)]
```

```
14 ser2 = xts(allSigmas[2,2,],time(logrets))[-c(1:20)]
```

```
15 ser12 = xts(allSigmas[1,2,],time(logrets))[-c(1:20)]
```

```
16
```

```
17 layout(matrix(c(1,2),nrow=2))
```

```
18 plot(sqrt(ser1),main="ABT Volatility")
```

```
19 plot(ser12/sqrt(ser1*ser2),main="ABT and CELG Corr")
```

1 **Exercise:** Comment on the form of the plots. Why did the correlation
2 between ABT and CELG drop suddenly in 2017?

3 The correlation dropped in 2017 because
4 CELG had a drug pulled from development

5

6

7

8

9

10

11

12

13

14

15

16

17

18

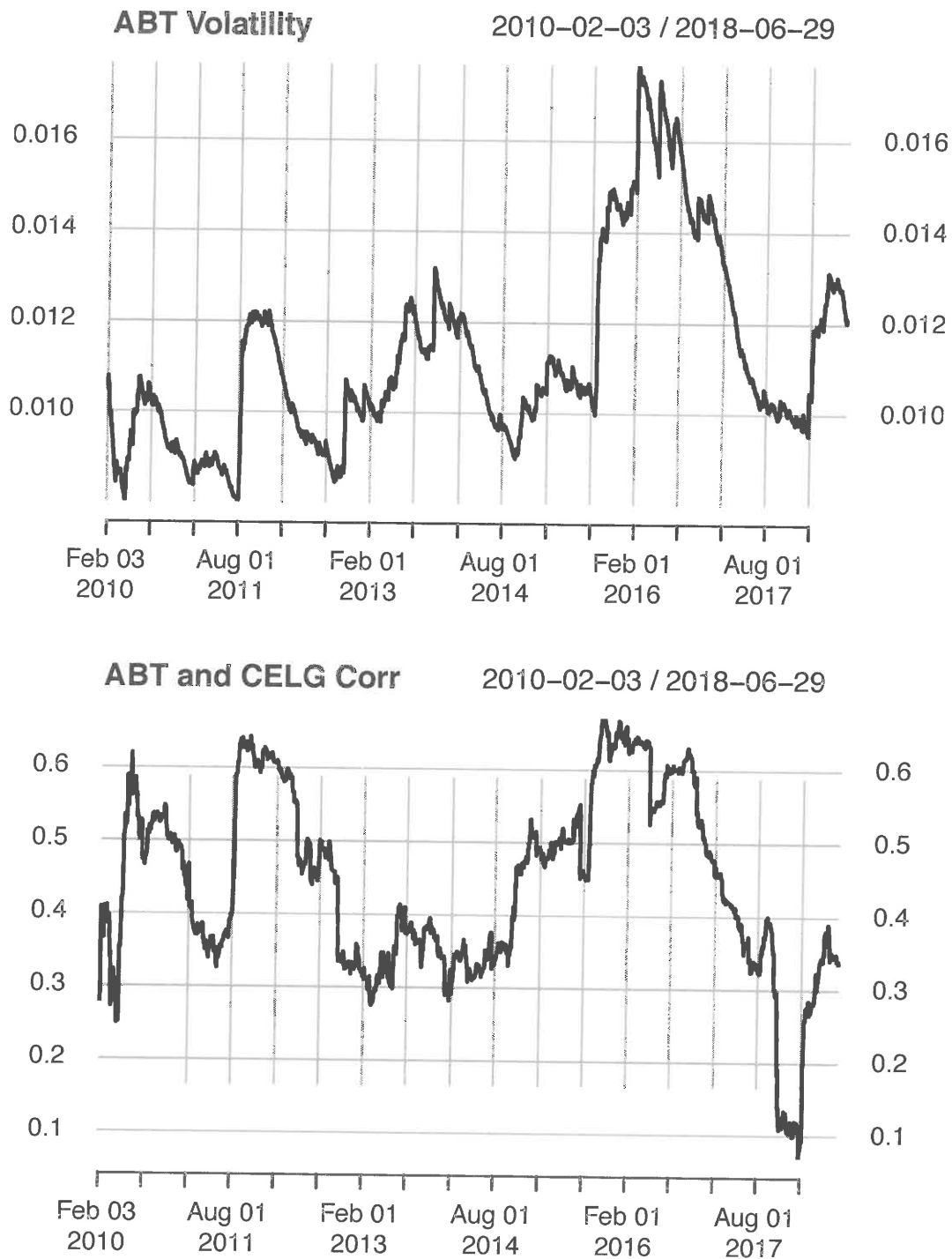


Figure 6: The evolution of the estimated volatility of ABT over time, along with the correlation between ABT and CELG.

Orthogonal GARCH Models

$$\underline{\tilde{X}}_t = \underline{\tilde{\mu}}_t + \underline{\tilde{a}}_t \quad \underline{\tilde{\Sigma}}_t = \text{Cov}(\underline{\tilde{a}}_t | \mathcal{F}_{t-1})$$

Assume that there exists a matrix $\underline{\tilde{U}}$ such that $\underline{\tilde{U}} \underline{\tilde{a}}_t$ transforms the data to $\underline{\tilde{Z}}_t$ which has cov. matrix $\underline{\tilde{I}}$

$$\begin{aligned} \text{Cov}(\underline{\tilde{Z}}_t) &= \text{Cov}(\underline{\tilde{U}} \underline{\tilde{a}}_t) = \underline{\tilde{U}} \text{Cov}(\underline{\tilde{a}}_t) \underline{\tilde{U}}^T = \underline{\tilde{I}} \\ &= \underline{\tilde{U}} \underline{\tilde{\Sigma}}_a \underline{\tilde{U}}^T \end{aligned}$$

$\underline{\tilde{U}}$ can be constructed by using a sample estimate of $\underline{\tilde{\Sigma}}_a$ and taking an appropriate decomp:

$$\underline{\tilde{U}} = \underline{\tilde{\Lambda}}^{-1/2} \underline{\tilde{V}}^T \leftarrow \text{eigenvectors of } \underline{\tilde{\Sigma}}_a$$

diagonal matrix with eigenvalues of $\underline{\tilde{\Sigma}}_a$

This forces $\underline{\tilde{U}} \underline{\tilde{\Sigma}}_a \underline{\tilde{U}}^T = \underline{\tilde{I}}$

Now, let $\underline{V}_t = \text{Cov}(\underline{Z}_t | \mathcal{F}_{t-1})$

Assume that the components of $\underline{Z}_t$ are also conditionally uncorrelated

So, can fit GARCH(p,q) model to each component of $\underline{Z}$ separately

$$\sigma_{i,t}^2 = \omega_i + \alpha_i Z_{i,t-1}^2 + \beta_i \sigma_{i,t-1}^2, \quad i=1, 2, \dots, k$$

e.g. fitting GARCH(1,1) to each

$$\underline{V}_t = \begin{bmatrix} \sigma_{1,t}^2 & & 0 \\ & \ddots & \\ 0 & & \sigma_{k,t}^2 \end{bmatrix}$$

$$\text{Then, } \underline{\Sigma}_t = \underline{U}^{-1} \underline{V}_t (\underline{U}^{-1})^T, \quad \underline{V}_t = \underline{U} \underline{\Sigma}_t \underline{U}^T$$

$$\hat{\underline{\Sigma}}_t = \hat{\underline{U}}^{-1} \hat{\underline{V}}_t (\hat{\underline{U}}^{-1})^T$$

Part 8: ARFIMA Models

Text references: §13.5 in Ruppert and Matteson

The $\text{ARIMA}(p, d, q)$ models which we considered earlier incorporated differencing into the $\text{ARMA}(p, q)$ model. The value of d in that model was a nonnegative integer chosen to create an approximately stationary series.

One can introduce *fractional differencing* to refine the choice of d .

Models which incorporate this are labelled $\text{ARFIMA}(p, d, q)$ or $\text{FARIMA}(p, d, q)$.

The advantage of these models is that they can model series with ACF which have slower decay than do typical ARMA models. Hence, these are used to model *long memory stationary processes*.

1 To see why this would be useful, consider the following situation.

2 Suppose that $\{Y_t\}$ is such that

3
$$\Delta^d Y_t = a_t$$

4 where $-0.5 < d < 0.5$ and $\{a_t\}$ is white noise.

5 Then:

6 • The series $\{Y_t\}$ is stationary.

7 • The ACF of $\{Y_t\}$ is

8
$$\rho_Y(h) = \frac{d(1+d) \cdots (h-1+d)}{(1-d)(2-d) \cdots (h-d)}, \quad h = 1, 2, \dots$$

9 • The PACF of $\{Y_t\}$ at lag h is $d/(h-d)$.

10 The critical fact is that the ACF is such that $\rho_Y(1) = d/(1-d)$ and

11
$$\rho_Y(h) \approx \frac{\Gamma(1-d)}{\Gamma(d)} h^{2d-1}.$$

12 Thus, the decay of the ACF is polynomial, instead of the typical ex-
13 ponential decay of the ARIMA ACF.

14 Hence, fractional differencing is used to model long memory pro-
15 cesses.

1 Next consider $\{a_t\}$ as the “remainder” from an ARIMA fit. For in-
2 stance, if

3
$$a_t = (1 + \theta B)^{-1}(1 - \phi B)\Delta(Y_t - \mu)$$



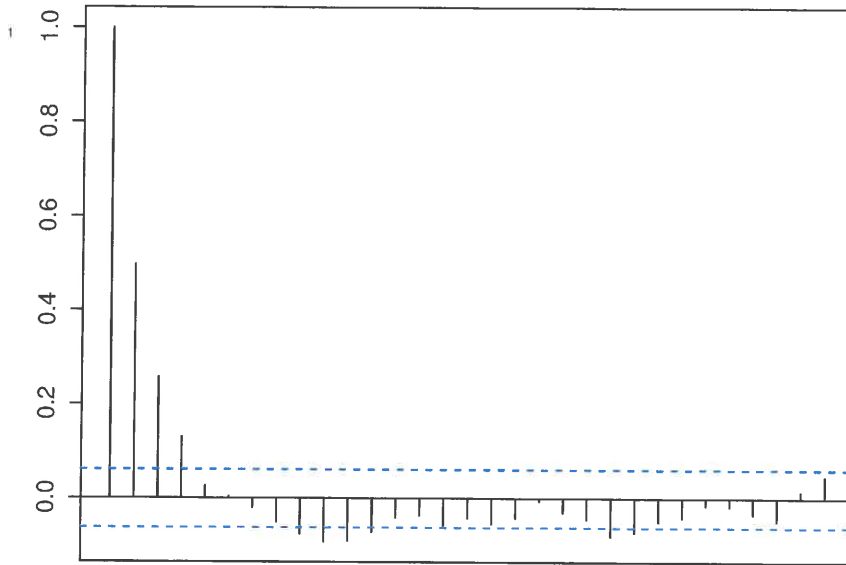
4 then we would have fit an ARIMA(1, 1, 1) model to $\{Y_t\}$, and $\{a_t\}$
5 would be the white noise component, as before.

6 If we find that $\{Y_t\}$ is stationary but has long memory, fractional dif-
7 ferencing may be appropriate:

8
$$a_t = (1 + \theta B)^{-1}(1 - \phi B)\Delta^{d+1}(Y_t - \mu)$$

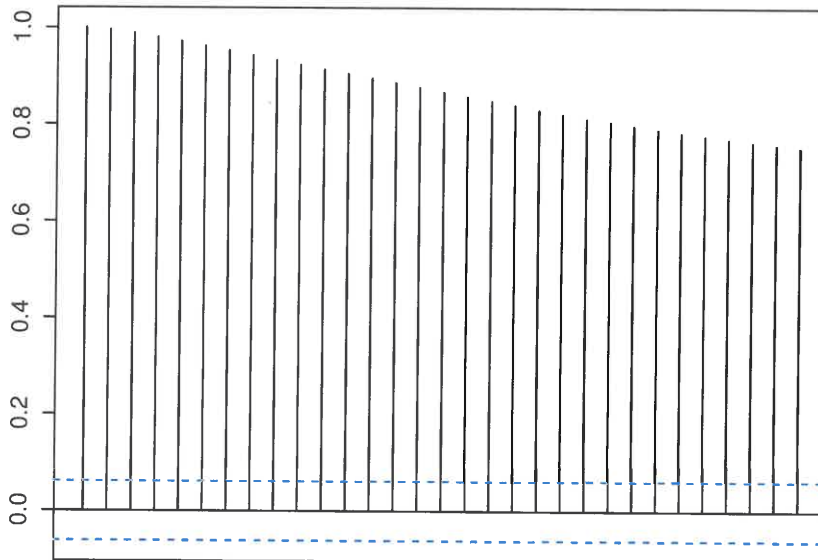
9 where $-0.5 < d < 0.5$.

10 Then, we say that $\{Y_t\}$ is an ARFIMA(1, $1 + d$, 1) process.



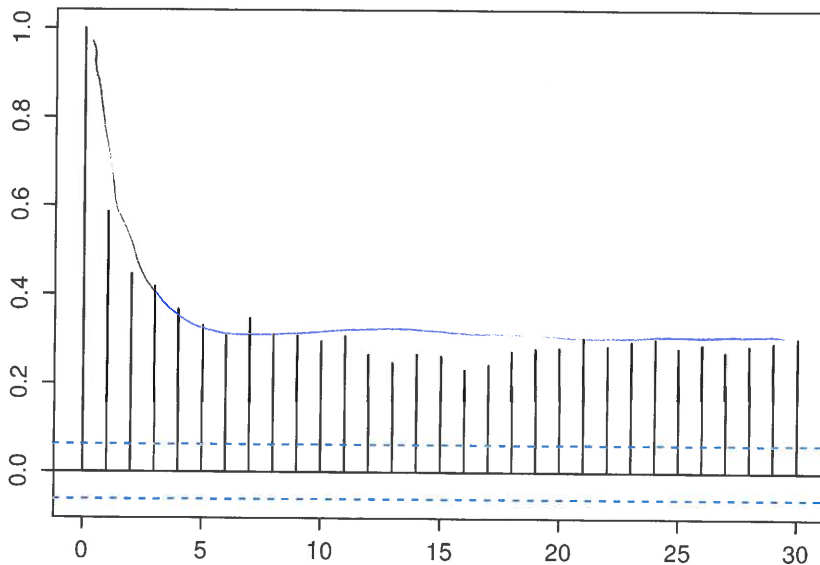
$AR(1)$

$$\phi = 0.5$$



$$X_t \approx X_{t-1} + \epsilon_t$$

$$X_t - X_{t-1} = \epsilon_t$$



$ARFIMA(1, 0.4, 0)$

"long memory process"

1 One way to think about this: One applies the “regular” amount of
2 (integer) differencing in order to obtain an approximately stationary
3 series. Denote this d_1 . Then, it is possible to “refine” this choice by
4 adding on an amount d_2 such that $-0.5 < d_2 < 0.5$. The “total”
5 amount of differencing is $d = d_1 + d_2$.

6 Keep in mind that $\Delta^d = \Delta^{d_1} \Delta^{d_2}$.

7 If $\Delta^d X_t$ is $\text{ARMA}(p, q)$, where d is not integer valued, then $\{X_t\}$ is
8 $\text{ARFIMA}(p, d, q)$.

1 **Comment:** The math behind “fractional differencing.”

2 Note that for $d = 0, 1, 2, \dots$

$$\begin{aligned}
 \Delta^d &= (1 - B)^d \\
 &= \sum_{k=0}^d \binom{d}{k} (-B)^k \\
 &= \sum_{k=0}^d \left[\frac{d!}{k!(d-k)!} \right] (-B)^k \\
 &= \sum_{k=0}^d \left[\frac{d(d-1) \cdots (d-k+1)}{k!} \right] (-B)^k \\
 &= \sum_{k=0}^d \left[\frac{\prod_{a=0}^{k-1} (d-a)}{k!} \right] (-B)^k \\
 &= \sum_{k=0}^{\infty} \left[\frac{\prod_{a=0}^{k-1} (d-a)}{k!} \right] (-B)^k
 \end{aligned}$$

“binomial theorem”
 $(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k$

15 This definition

$$\Delta^d = \sum_{k=0}^{\infty} \left[\frac{\prod_{a=0}^{k-1} (d-a)}{k!} \right] (-B)^k$$

17 can be extended to a general (non-integer) choice of d . In practice,

18 we only need to be concerned with $-0.5 < d < 0.5$.

1 ARFIMA in R

2 ARFIMA models can be fit in R using the function `arfima()` from
3 package `arfima`.

4 The value of d is best thought of an additional *parameter* that requires
5 estimation. So, if one specifies the argument `order=c(p, d, q)`, there
6 are now $p + q + 1$ parameters which are estimated via maximum like-
7 lihood.

8 The `summary()` output of `arfima()` includes the value of AIC,
9 which can be used for choosing p and q .

- 1 **R Exercise:** Obtain the monthly inflation rate for the United States
 2 from the period Feb. 1950 to Dec. 1990 from the R package Ecdat:

```
3 library(Ecdat)
4 data(Mishkin)
5 monthlyinfl = Mishkin[,1]
```

6 Now, consider the following:

- 7 1. Does the raw series appear to be stationary? What happens when
 8 you take the first differences?

*Based on ADF test, seems stationary.
 ACF of first differenced series
 shows evidence of over differencing.*

- 9 2. Look at the ACF of the raw series and comment.

ACF suggests long memory

- 10 3. Run the search performed below

```
11 maxp = 5
12 maxq = 5
13 holdaic = matrix(0, nrow=(maxp+1), ncol=(maxq+1))
14 for(p in 0:maxp)
15 {
16   for(q in 0:maxq)
17   {
18     holdaic[(p+1), (q+1)] =
19       summary(arfima(ts(monthlyinfl),
20                        order=c(p, 0, q), quiet=TRUE))$aic
21   }
22 }
```

- 23 Which model is best, as judged by AIC?

*$p=2, q=4, \hat{d}=0.29$, ARFIMA(2, 0.29, 4)
 model*

Part 9: Revisiting Regression

Recall that the classic normal linear regression model can be written as

$$Y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \epsilon_i$$

where the ϵ_i are assumed to be iid normal with mean zero and variance σ^2 .

We discussed dealing with deviations from normality, but in cases where i indexes time, a common problem is that there will be significant serial correlation in the model error.

Exercise: What would be a simple, graphical approach to diagnosing serial correlation in the model error? (ϵ_t)

Construct ACF of the residuals. Significant serial correlation in ϵ_t should be evident in this plot.

1 Another tool is the *Durbin-Watson Test*. This is hypothesis test de-
2 signed specifically to test for serial correlation in the residuals from
3 a linear model.

4 This test is implemented as `durbinWatsonTest()` as part of pack-
5 age `car`. The use of this function is simple: One provides the output
6 from `lm()` along with the maximum lag at which correlation is to be
7 tested. An example of the output is shown below.

```
8 > holdlm = lm(y~x)
9 > durbinWatsonTest(holdlm, 5)
10 lag Autocorrelation D-W Statistic p-value
11 1 0.190213104 1.569622 0.020
12 2 0.022955459 1.858580 0.534
13 3 -0.024442893 1.950965 0.946
14 4 -0.047906415 1.968193 0.898
15 5 -0.040479939 1.929293 0.932
16 Alternative hypothesis: rho[lag] != 0
```

1 If the residuals appear to be nonstationary, then differencing can be
2 applied to both the response and the predictors.

3 If it is determined that significant serial correlation is present, then
4 it can be taken into account by fitting an $\text{ARIMA}(p, d, q)$ time series
5 model in place of the model error term.

6 We could write our model as follows:

7
$$\Delta^d Y_t = \Delta^d (\beta_0 + \beta_1 x_{t1} + \cdots + \beta_p x_{tp}) + \epsilon_t$$

8 where

9
$$(1 - \phi_1 B - \cdots - \phi_p B^p) \epsilon_t = (1 + \theta_1 B + \cdots + \theta_q B^q) u_t$$

10 and $\{u_t\}$ is assumed to be white noise. (If we use `arima()` for fitting,
11 it will be assumed that the u_t are iid normal.)

12 Now our regression model includes $p + q$ additional parameters.

13 These, along with the β parameters, can be estimated using maxi-
14 mum likelihood.

- 1 In R, such a model can be fit using `arima()`. The vector of responses
- 2 in the regression is passed in as the time series, while the argument
- 3 `xreg` is used to specify the predictors.

- 4 By specifying `order=c(1,1,1)`, the appropriate first differencing
- 5 will be applied to the predictors and the response.

- 6 Note that `xreg` must be a matrix, with each column being a single
- 7 predictor. (A column is not needed for the intercept.) Use the func-
- 8 tion `cbind()` to join together the predictors into a matrix.

- 9 The function `auto.arima()` can also be used, with identical syntax.

1 **Example: US Treasury Rates**

2 This example is based on Section 2.9 in Tsay.

3 In this example we will fit a regression where the response is the 3-
4 year Treasury constant maturity rate, while the predictor is the 1-year
5 Treasury constant maturity rate.

6 Below is the code for obtaining the data, and then fitting three differ-
7 ent models. The first is a simple linear regression model. The second
8 is a model which assumes the errors are ARIMA(1,1,1). In the third
9 model, the order of the ARMA process is chosen using AICc.

```
10 OneYearTreas = as.numeric(getSymbols("WGS1YR",  
11     src="FRED", auto.assign=F))  
12 ThreeYearTreas = as.numeric(getSymbols("WGS3YR",  
13     src="FRED", auto.assign=F))  
14  
15 holdfit1 = lm(ThreeYearTreas ~ OneYearTreas)  
16 holdfit2 = arima(ThreeYearTreas, order=c(1,1,1),  
17     xreg = OneYearTreas)  
18 holdfit3 = auto.arima(ThreeYearTreas,  
19     xreg = OneYearTreas, d=1)
```

3 **R Exercise:** Address the following questions using the example on
2 the previous page:

4 1. Is there a clear relationship between the response and predictor
5 in this case? Clear linear relationship

6 2. Is there a need to include a time series model for the model error? Yes,
7 Do the residuals appear to be stationary? Not stationary.

8 3. Is the ARIMA(1,1,1) sufficient? ACF suggests possibly more
serial correlation.

9 4. What order of model is chosen when `auto.arima()` is used?
ARIMA(5,1,3)

10 5. Is there still evidence of correlation in the residuals when the
11 model chosen by `auto.arima()` is utilized? Seems as if it
has been removed.

Regression with ARMA/GARCH Errors

Unfortunately, function `garchFit()` does not possess the capacity to incorporate predictor variables as simply as does `arima()` and `auto.arima()`.

The function `ugarchspec()` that is part of the `rugarch` package includes an `external.regressors` argument that allows one to include predictors.

Or, one can use three-step process to incorporate such a model into a regression:

1. Estimate the β parameters in the linear model using standard least squares.
2. Fit a time series model to the residuals from the model fit above.
3. Refit the model using *weighted least squares* with weights equal to the reciprocals of the conditional variances found in step 2.

We previously found the conditional variances from `garchFit()`

```
> holdfit@sigma.t
```

Hence, the model fit in step 3 would include the argument

```
weights = 1/(holdfit@sigma.t^2)
```


- 1 **R Exercise:** Refit the above model, but include a ARMA/GARCH
- 2 component onto the fit to the model error.

Should first difference the predictor and response prior to taking the three steps above